Personal Finance Tracker

Week 4 Final Project — Python File Handling & Modular Design

---

1. Project Overview

A complete command-line personal finance tracker built in Python. It lets you log expenses, categorize spending, set monthly budgets, generate reports, and keep your data safe with automatic backups — all built on nothing but the Python standard library.

This is the Week 4 capstone project, combining file handling, error handling, context managers, and multi-module code organization from Weeks 1–4.

What it does:

- Add, view, search, edit, and delete expenses
- Categorize every expense (Food, Transport, Entertainment, Bills, Shopping, Health, Education, Other)
- Save all data to JSON, with automatic timestamped backups before every write
- Recover automatically from a backup if the main data file is ever corrupted
- Export expenses to CSV for use in Excel / Google Sheets
- Set a monthly budget and see at a glance whether you're under or over it
- Generate a monthly report, a category breakdown (text bar chart), a multi-month spending trend, and overall statistics

2. Setup Instructions

Requirements: Python 3.8 or later. No external packages needed.

```
# 1. Clone or download this folder
cd week4-finance-tracker

# 2. (Optional) create a virtual environment
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

# 3. Install dependencies (there are none, but this confirms the file works)
pip install -r requirements.txt

# 4. Run the app
python run.py
```

The first time you run it, `data/expenses.json` is created automatically (a small sample dataset is already included so the reports have something to show immediately).

3. Code Structure

```
week4-finance-tracker/
|-- finance_tracker/             (the application package)
|   |-- __init__.py
|   |-- main.py                  menu system / controller - wires everything together
|   |-- expense.py               Expense data model + validation
|   |-- expense_manager.py       in-memory collection: add/remove/search/budgets
|   |-- file_handler.py          ALL file I/O: JSON load/save, CSV export/import, backups
|   |-- reports.py               turns data into readable reports & text charts
|   `-- utils.py                 reusable, self-validating input prompts for the CLI
```

```
|-- data/
|   |-- expenses.json            main data file (sample data included)
|   |-- backup/                  automatic timestamped backups land here
|   `-- exports/                 CSV exports land here
|-- tests/
|   |-- test_expense.py          Expense + ExpenseManager unit tests
|   |-- test_file_handler.py     persistence, backup/restore, CSV tests (temp dirs)
|   `-- test_reports.py          report-generation tests
|-- requirements.txt
|-- README.md
|-- .gitignore
`-- run.py                       entry point: python run.py
```

Why it's organized this way: each module has exactly one job. `expense.py` only knows how to validate and represent one expense. `expense_manager.py` only knows how to manage a collection of them in memory. `file_handler.py` is the only module that ever opens a file. `reports.py` only turns data into text. `main.py` is the thin layer that connects user input to all of the above. This means any module can be tested and modified independently — see the test suite in Section 7.

4. Technical Details

Data model & validation (expense.py)

Every Expense validates itself on creation: dates must match YYYY-MM-DD, amounts must be a positive number, and categories must be one of a fixed list. Invalid data raises `ExpenseValidationError` instead of silently corrupting the dataset.

File handling (file_handler.py)

- Every file operation uses a `with` block, so files are always closed properly even if an error occurs mid-operation.
- **Writes are backed up first:** before `save_data()` overwrites expenses.json, it copies the current file into data/backup/ with a timestamp.
- **Atomic writes:** data is written to a temporary .tmp file first, then swapped into place with `os.replace()`, so a crash mid-write can never leave expenses.json half-written.
- **Automatic recovery:** if expenses.json is ever corrupted (invalid JSON), `load_data()` automatically attempts to restore the most recent backup instead of crashing or wiping the dataset.
- Errors handled explicitly: missing files, permission errors, corrupted JSON, malformed CSV — each has its own except branch with a clear message instead of a raw traceback.

Reports (reports.py)

No plotting library is needed — category breakdowns and spending trends are rendered as text bar charts (block characters scaled to the largest value), which is dependency-free and readable directly in a terminal.

Modular menu system (main.py)

The menu maps each numeric choice to a bound method via a dictionary (`menu_actions`), rather than a long if/elif chain, making it easy to add new menu items.

5. Features Checklist

- ✓ File operations for data persistence (JSON + CSV)

- ✓ Modular code structure across separate files
- ✓ Comprehensive error handling for file operations
- ✓ Data validation for all inputs (dates, amounts, categories, budgets)
- ✓ Reports with statistics and text-based visualizations
- ✓ Multiple expense categories
- ✓ Search and filter functionality (keyword, category, date range, amount range)
- ✓ User-friendly numbered menu system
- ✓ Backup and data recovery features (manual + automatic)
- ✓ Budget setting and tracking, with over/under-budget alerts
- ✓ Edit and delete existing expenses
- ✓ CSV export (and a CSV import helper for future use)

6. How to Run

```
cd week4-finance-tracker
python run.py
```

Sample session

```
============================================================
          PERSONAL FINANCE TRACKER
============================================================

========================================
              MAIN MENU
========================================
1. Add New Expense
2. View All Expenses
3. Search Expenses
4. Generate Monthly Report
5. View Category Breakdown
6. Set/Update Budget
7. Export Data to CSV
8. View Statistics
9. Backup/Restore Data
10. View Spending Trend
11. Edit/Delete an Expense
0. Save & Exit
========================================

Enter your choice (0-11): 5

--- CATEGORY BREAKDOWN ---
Specific month? (y/n): n
CATEGORY BREAKDOWN (all time)
==============================
Food            Rs.    450.00  [##############################]
Entertainment   Rs.    899.00  [##############################]
Transport       Rs.    120.00  [####]
```

Visual documentation note: add your own terminal screenshots here before submitting (e.g. add-expense flow, a generated report, and the category breakdown chart) to satisfy the 'Visual Documentation' grading requirement.

7. Testing Evidence

Unit tests cover the data model, the manager, file persistence (including corrupted-file recovery, using temporary directories so tests never touch real data), and report generation.

```
python -m unittest discover -s tests -v
```

Result on this submission: 31 tests, all passing.

```
Ran 31 tests in 0.011s

OK
```

Example test cases included:

- Rejects a negative or zero expense amount
- Rejects an invalid date format or invalid category
- Loading a file with a corrupted row skips only that row, not the whole dataset
- Saving twice in a row produces a backup of the first save
- A corrupted expenses.json with no backup available falls back to an empty (not crashed) dataset
- Monthly report correctly flags over-budget vs. within-budget months

8. Notes on File Formats

File	Purpose
`data/expenses.json`	Single JSON object with two keys: "expenses" (list of expense records) and "budgets" (dict keyed by YYYY-MM). Kept as JSON because the data has nested / optional structure.
`data/exports/*.csv`	Flat id, date, amount, category, description rows, timestamped, for opening in Excel / Sheets.
`data/backup/*.json`	Full timestamped snapshots of expenses.json, created automatically before every save.

Built as the Week 4 capstone project — file handling, error handling, and modular project structure applied to a real, working application.